РО 1.3. Базовое администрирование Linux

Лекция . Сеть Linux, SSH и базовый firewall

Цель лекции

- Понять, какие сетевые параметры нужны Linux-серверу для работы в сети.
- Научиться проверять интерфейсы, IP-адреса, маршруты и DNS.
- Научиться объяснять работу SSH, SSH-ключей и базовой настройки sshd.
- Научиться включать firewall так, чтобы не потерять удаленный доступ.

Список учебных вопросов

1. Сетевые параметры Linux: IP-адрес, mask/prefix, gateway, DNS, hostname.
2. Просмотр сетевых интерфейсов: ip addr, ip link.
3. Проверка маршрутов: ip route.
4. Проверка DNS: /etc/resolv.conf, resolvectl, dig/nslookup на уровне назначения.
5. Настройка hostname через hostnamectl.
6. SSH: назначение, клиент, сервер, порт 22.
7. SSH-ключи: назначение открытого и закрытого ключа.
8. Базовая настройка sshd_config: парольный вход, root login, доступ по ключу.
9. Базовый firewall через ufw: разрешение SSH/HTTP и ограничение доступа по сети.

1. Сетевые параметры Linux

Linux-серверу нужны базовые сетевые параметры, чтобы быть доступным в сети.

IP-адрес определяет сам узел в сети.

Маска или prefix определяет границы локальной сети.

Gateway или шлюз по умолчанию нужен для выхода в другие сети.

DNS нужен для преобразования имен в IP-адреса, hostname - для имени самого узла.

СХЕМА: СЕТЕВЫЕ ПАРАМЕТРЫ LINUX-СЕРВЕРА

Ключевые параметры, файлы конфигурации и команды для настройки сети в Linux.

Сетевые параметры определают, как сервер подключается к сети и взаимодействует с другими устройствами и Интернетом.

1. СЕТЕВАЯ ТОПОЛОГИЯ (ПРИМЕР)



2. СЕТЕВЫЕ ИНТЕРФЕЙСЫ

Интерфейс	Тип	Описание
lo	Loopback	Локальный интерфейс (127.0.0.1)
eth0 / ens*	Ethernet	Проводное подключение
wlan0	Wi-Fi	Беспроводное подключение
bond0	Bonding	Агрегация каналов
docker0	Bridge	Виртуальный мост (Docker)
tun0	TUN/TAP	VPN-интерфейсы

3. IP-АДРЕСАЦИЯ

IPv4-адрес:	192.168.1.100
Маска подсети:	255.255.255.0 (/24)
Шлюз по умолчанию:	192.168.1.1
DNS-серверы:	8.8.8.8, 1.1.1.1
Broadcast:	192.168.1.255
Сеть:	192.168.1.0

4. ОСНОВНЫЕ ФАЙЛЫ КОНФИГУРАЦИИ

/etc/netplan/*.yaml	Netplan (Ubuntu 18.04+)
/etc/network/interfaces	ifupdown (Debian/Ubuntu старые)
/etc/sysconfig/network-scripts/	Network Scripts (RHEL/CentOS 7-)
/etc/NetworkManager/system-connections/	NetworkManager (профили)
/etc/resolv.conf	DNS-настройки
/etc/hosts	Локальное сопоставление имён

5. НАСТРОЙКА СЕТИ (ПРИМЕРЫ)

Netplan (Ubuntu 18.04+)

```
# /etc/netplan/01-netcfg.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: no
      addresses: [192.168.1.100/24]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

```
sudo netplan generate
sudo netplan apply
```

ifupdown (Debian/Ubuntu старые)

```
# /etc/network/interfaces
auto ens33
iface ens33 inet static
    address 192.168.1.100
    netmask 255.255.255.0
    gateway 192.168.1.1

    dns-nameservers 8.8.8.8 1.1.1.1
```

```
sudo systemctl restart networking
```

Network Scripts (RHEL/CentOS 7-)

```
# /etc/sysconfig/network-scripts/ifcfg-ens33
TYPE=Ethernet
BOOTPROTO=none
NAME=ens33
DEVICE=ens33
ONBOOT=yes
IPADDR=192.168.1.100
PREFIX=24
GATEWAY=192.168.1.1
DNS1=8.8.8.8
DNS2=1.1.1.1
```

```
sudo systemctl restart network
```

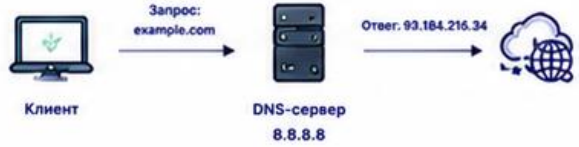
6. ПРОВЕРКА ИНФОРМАЦИИ О СЕТИ

ip addr	Показать IP-адреса интерфейсов
ip route	Таблица маршрутизации
ip link	Состояние сетевых интерфейсов
ss -tuln	Открытые порты и службы
resolvectl status	Состояние DNS (systemd-resolved)
ping 8.8.8.8	Проверка соединения
traceroute 8.8.8.8	Маршрут до хоста
hostname -I	Показать IP-адреса сервера
nmcli dev status	Статус устройств (NetworkManager)

7. DNS-НАСТРОЙКИ

/etc/resolv.conf (пример)

```
nameserver 8.8.8.8
nameserver 1.1.1.1
search example.local
options timeout:2 attempts:2
```



8. МАРШРУТИЗАЦИЯ

Добавить маршрут (временно):

```
sudo ip route add 10.10.0.0/24 via 192.168.1.1 dev ens33
```

Добавить маршрут (постоянно) в Netplan:

```
routes:
  - to: 10.10.0.0/24
    via: 192.168.1.1
```

Проверка маршрутов:

```
ip route
```

Таблица маршрутизации (пример)

Назначение	Шлюз	Интерфейс
default	192.168.1.1	ens33
192.168.1.0/24	—	ens33
10.10.0.0/24	192.168.1.1	ens33

9. БЕЗОПАСНОСТЬ СЕТИ

- Фаервол (UFW / firewalld / nftables)
- Ограничивайте доступ только к нужным портам.
- Отключайте неиспользуемые сервисы и интерфейсы.
- Используйте SSH-ключи, а не пароли.
- Регулярно обновляйте систему и пакеты.

10. ПОЛЕЗНЫЕ КОМАНДЫ

Управление сетью (systemd-networkd)

```
sudo systemctl restart systemd-networkd
sudo systemctl status systemd-networkd
networkctl status
networkctl list
```

NetworkManager (nmcli)

```
nmcli dev status
nmcli con show
nmcli con up "Имя_профиля"
nmcli con down "Имя_профиля"
```

ДОПОЛНИТЕЛЬНО

Полезные порты

22/tcp	SSH
80/tcp	HTTP
443/tcp	HTTPS
53/tcp	DNS

Типы IP-адресов

192.168.x.x	Частные (LAN)
10.x.x.x	Частные (LAN)
172.16.x.x – 172.31.x.x	Частные (LAN)
127.0.0.1	Loopback

Способы получения IP

- DHCP – автоматически от сервера
- Static – вручную (статический IP)
- PPPoE – через интернет-провайдера

Важные службы

- systemd-networkd – управление сетью
- NetworkManager – удобное управление
- systemd-resolved – DNS-кэш и резолвер

ВАЖНО!

- Неправильные сетевые настройки могут привести к потере доступа к серверу.
- Всегда проверяйте конфигурацию перед применением.
- Используйте резервный доступ (консоль, IPMI, облачный доступ).

IP-адрес и prefix

IP-адрес - числовой адрес узла в IP-сети.

Prefix показывает, какая часть адреса относится к сети.

Пример 192.168.10.20/24 означает адрес 192.168.10.20 в сети 192.168.10.0/24.

/24 соответствует маске 255.255.255.0.

Если адрес или prefix указаны неверно, сервер может оказаться в неправильной сети.

Gateway и default route

Gateway - маршрутизатор, через который сервер выходит за пределы своей локальной сети.

Default route - маршрут по умолчанию для всех неизвестных сетей.

Если gateway не задан, сервер может видеть соседей в LAN, но не интернет или другие подсети.

Типичная запись default route: default via 192.168.10.1 dev ens33.

DNS и hostname

DNS - Domain Name System, система преобразования имен в IP-адреса.

Без DNS можно подключаться по IP, но имена сайтов и серверов не будут разрешаться.

Hostname - имя самого Linux-узла.

Корректный hostname важен для журналов, SSH, мониторинга и документации.

Пример адресного плана учебного сервера

- Имя сервера: linux-srv01.
- IP-адрес: 192.168.10.20/24.
- Gateway: 192.168.10.1.
- DNS: 192.168.10.1 или 8.8.8.8 в учебном стенде.
- SSH-порт: TCP 22.

2. Просмотр сетевых интерфейсов

Сетевой интерфейс - точка подключения Linux к сети.

Физический или виртуальный адаптер может называться **ens33**, **enp0s3**, **eth0**, **wlan0**.

Перед настройкой адреса нужно понять, какой интерфейс существует и включен ли он.

Для этого используют команды семейства **ip**.

ip link: состояние интерфейса

Для просмотра интерфейсов используют: **ip link**.

В выводе важны имя интерфейса, состояние UP/DOWN и MAC-адрес.

UP означает, что интерфейс включен на уровне системы.

DOWN означает, что интерфейс выключен или не поднят.

Если интерфейс DOWN, IP-настройка может быть корректной, но связи не будет.

Пример чтения ip link

Строка вида:

2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP>

показывает интерфейс ens33.

UP означает административно включен.

LOWER_UP означает, что есть линк на физическом или виртуальном уровне.

link/ether показывает MAC-адрес сетевого адаптера.

Если LOWER_UP отсутствует, нужно проверить виртуальный адаптер, кабель или режим сети ВМ.

ip addr: IP-адреса интерфейсов

Для просмотра IP-адресов используют: ip addr.

Короткий вариант: **ip a**.

В выводе важны inet для IPv4 и inet6 для IPv6.

Пример:

inet 192.168.10.20/24 brd 192.168.10.255 scope global ens33.

Если у интерфейса нет inet-адреса, сервер не настроен в IPv4-сети.

Проверка конкретного интерфейса

Если интерфейсов много, удобно смотреть один интерфейс.

Пример: ip addr show ens33.

Так проще увидеть адрес, prefix и состояние именно нужного адаптера.

После настройки адреса эта команда подтверждает, что адрес применился.

3. Проверка маршрутов

Маршрут определяет, куда Linux отправляет IP-пакеты.

Локальная сеть обычно доступна напрямую через интерфейс.

Другие сети доступны через gateway.

Если маршрут неверный, IP-адрес на сервере может быть правильным, но связи с внешними сетями не будет.

ip route: таблица маршрутизации

Для просмотра маршрутов используют: ip route.

Строка default via 192.168.10.1 dev ens33 показывает gateway по умолчанию.

Строка 192.168.10.0/24 dev ens33 показывает локальную сеть интерфейса.

src 192.168.10.20 показывает адрес источника, который будет использоваться.

Проверка маршрута до адреса

Чтобы понять, куда пойдет пакет, используют:

ip route get 8.8.8.8.

Команда показывает выбранный gateway, интерфейс и адрес источника.

Это полезнее, чем гадать по всей таблице маршрутов.

Если gateway отсутствует, команда покажет ошибку или только локальный маршрут.

ping gateway и внешний адрес

Сначала проверяют доступность gateway: ping -c 4 192.168.10.1.

Если gateway не отвечает, проблема часто на уровне LAN, адресации или виртуальной сети.

Затем проверяют внешний IP: ping -c 4 8.8.8.8.

Если gateway отвечает, а внешний IP нет, нужно проверять маршрутизацию и NAT.

СХЕМА: LINUX-СЕРВЕР. ДИАГНОСТИКА СЕТИ ПО УРОВНЯМ

Пошаговый подход: от приложения до физического уровня.

Диагностика по уровням помогает быстро локализовать проблему и выбрать правильные инструменты.

УРОВЕНЬ	ЧТО ПРОВЕРЯЕМ	ИНСТРУМЕНТЫ (Linux)	ПРИМЕРЫ КОМАНД	ЧТО СМОТРИМ / ИНТЕРПРЕТАЦИЯ	ПРИМЕР ПРОБЛЕМЫ

7 ПРИКЛАДНОЙ (Приложения)
Сервисы, протоколы
HTTP, DNS, SMTP, SSH и др.
	○ Работает ли сервис?
	○ Доступность приложения
	○ DNS-имена, HTTP-ответы
		curl, wget
		dig, nslookup, host
		ssh, telnet, nc
			`curl -I https://example.com`
			`dig example.com`
			`ssh user@192.168.1.10`
			`nc -vz example.com 80`
				✓ Код ответа 200 OK
				✓ Резолвится имя в IP
				✓ Сервис отвечает
					⚠ Сервис недоступен
					Таймауты приложений
					DNS не резолвится

4-6 ТРАНСПОРТНЫЙ (Сеансы, представление)
TCP, UDP
Порты, соединения
	○ Открыт ли порт?
	○ Устанавливается ли соединение?
	○ Потеря пакетов, ошибки TCP/UDP
		ss, netstat
		nc (netcat)
		lsof, conntrack
			`ss -tulnp`
			`nc -vz 192.168.1.10 22`
			`ss -ti state established`
			`sudo conntrack -L`
				✓ LISTEN на нужном порту
				✓ ESTABLISHED соединения
				✓ Нет ошибок и потерь
					⚠ Порт закрыт / firewall
					Отказ в соединении
					Потеря пакетов

3 СЕТЕВОЙ (Сеть)
IP-адресация, маршруты, ICMP
	○ Доступность узла по IP
	○ Маршрутизация
	○ ICMP ошибки
		ip, ping, traceroute
		mtr, tracepath
		ip route, ip rule
			`ping 8.8.8.8`
			`traceroute 8.8.8.8`
			`mtr -rw 8.8.8.8`
			`ip route show`
				✓ Ответы на ping
				✓ Маршрут без петель
				✓ Нет ICMP ошибок
					⚠ Нет маршрута
					Узел не пингуется
					ICMP unreachable

2 КАНАЛЬНЫЙ (Канальный)
Ethernet, MAC-адреса, VLAN, ARP
	○ Связь в локальной сети
	○ ARP-таблица
	○ VLAN, MAC-адреса
		ip link, ip addr
		ip neigh (arp)
		bridge, vlan, tcpdump
			`ip addr show`
			`ip neigh show`
			`tcpdump -i eth0 arp`
			`bridge vlan show`
				✓ MAC-адрес найден
				✓ ARP без ошибок
				✓ Пакеты проходят
					⚠ ARP не отвечает
					Неверный VLAN
					Пакеты не на том сегменте

1 ФИЗИЧЕСКИЙ (Физический)
Кабель, разъёмы, скорость, сигнал
	○ Линк активен?
	○ Скорость и дуплекс
	○ Ошибки интерфейса
		ethtool
		ip -s link
		dmesg, lshw
			`ethtool eth0`
			`ip -s link show eth0`
			`dmesg | grep -i eth`
				✓ LINK DETECTED: yes
				✓ Скорость и дуплекс OK
				✓ Ошибки RX/TX = 0
					⚠ Кабель отключён
					Линк down
					Много ошибок / дропов

ПОШАГОВЫЙ АЛГОРИТМ ДИАГНОСТИКИ

1. **Проверка приложения**
   Проверьте доступность сервиса и разрешение DNS.
   `curl, dig, nc`
2. **Проверка соединения**
   Убедитесь, что порт открыт и соединение устанавливается. `ss, nc`
3. **Проверка сети**
   Проверьте доступность узла и маршрут.
   `ping, traceroute, ip route`
4. **Проверка локальной сети**
   Проверьте ARP, MAC, VLAN, интерфейсы.
   `ip neigh, tcpdump, bridge`
5. **Проверка физики**
   Проверьте линк, кабель, ошибки интерфейса.
   `ethtool, ip -s link`

ПОЛЕЗНЫЕ ФАЙЛЫ И ЛОГИ

`/var/log/syslog`	– системные сообщения
`/var/log/messages`	– системные сообщения
`/var/log/kern.log`	– сообщения ядра
`/var/log/secure`	– аутентификация
`journalctl -xe`	– системный журнал (systemd)

БЫСТРЫЙ «ЧЕК-ЛИСТ»

- ☑ Сервис работает и доступен?
- ☑ Порт открыт и соединение установлено?
- ☑ Маршрут до узла корректный?
- ☑ ARP и MAC-адреса корректны?
- ☑ Интерфейс UP и без ошибок?

ЧАСТЫЕ ПРОБЛЕМЫ И ПРИЧИНЫ

Порт недоступен	Firewall / сервис не запущен
Не пингуется узел	Нет маршрута / фильтрация ICMP
Обрыв соединения	Потеря пакетов / MTU / дропы
Нет ARP-ответа	Проблема L2 / VLAN / кабель
Линк down	Кабель / порт / питание

ПОЛЕЗНЫЕ КОМАНДЫ «НА ВСЕ СЛУЧАИ»

Интерфейсы и IP

```
ip a
ip -br a
ip link show
```

Маршруты

```
ip route show
ip rule show
```

Состояние соединений

```
ss -tulnp
ss -ti
conntrack -L
```

Диагностика сети

```
ping -c 4 <host>
traceroute <host>
mtr -rw <host>
```

Захват пакетов

```
tcpdump -i eth0 -nn
tcpdump -i eth0 port 80
```

СОВЕТ

Всегда двигайтесь сверху вниз по уровням. Как только найден сбой — устраняйте причину и перепроверяйте выше.

4. Проверка DNS

DNS проверяют отдельно от IP-связности.

Если ping 8.8.8.8 работает, но ping google.com нет, вероятна проблема DNS.

DNS-клиент Linux получает адреса DNS-серверов из сетевой настройки или systemd-resolved.

Важно отличать проблему маршрута от проблемы разрешения имен.

Как работает DNS в Debian (Кратко)

Когда программа (например, ping или веб-сервер) запрашивает адрес google.com, операционная система действует по цепочке:

- Файл /etc/hosts: Сначала система заглядывает сюда. Если там вручную прописано соответствие IP и домена, запрос завершается.
- Системный резолвер (Resolver): Если в /etc/hosts ответа нет, специальная библиотека ядра отправляет UDP-запрос на IP-адреса DNS-серверов (например, 8.8.8.8), которые указаны в конфигурационных файлах системы (/etc/resolv.conf).

Файл /etc/hosts

Файл /etc/hosts — это простой текстовый файл в Linux. Он связывает IP-адреса с доменными именами.

Когда вы вводите в терминале или браузере имя (например, server1), операционная система в первую очередь заглядывает в файл /etc/hosts.

Если она находит там это имя, то сразу берёт указанный IP-адрес и не обращается к глобальной сети DNS.

Если имени там нет, система идёт дальше — к внешним DNS-серверам (например, к 8.8.8.8).

Каждая строка — это одна запись, которая строится по очень простой схеме:

[IP-адрес] [Имя_компьютера] [Альтернативное_имя_или_алиас]

```
127.0.0.1   localhost
::1         localhost ip6-localhost ip6-loopback
192.168.1.10 srv-web-01 web
```

/etc/resolv.conf

/etc/resolv.conf традиционно содержит настройки DNS-клиента.

В современных системах это часто символическая ссылка на файл systemd-resolved.

Просмотр: **cat /etc/resolv.conf**.

Строка nameserver показывает DNS-сервер.

Файл лучше не редактировать вручную, если системой управляет NetworkManager или systemd-resolved.

```
nameserver 8.8.8.8
nameserver 1.1.1.1
```

dig и nslookup

dig и **nslookup** используют для явной проверки DNS-запросов.

Пример: **dig example.com**

Короткий ответ: **dig +short example.com**

nslookup example.com показывает адреса и DNS-сервер, который отвечал.

Если утилит нет, их обычно устанавливают пакетами dnsutils или bind9-dnsutils.

СХЕМА: КАК LINUX РАЗРЕШАЕТ DNS-ИМЯ



КЛЮЧЕВЫЕ ФАЙЛЫ

- **/etc/hosts** — локальные имена
- **/etc/nsswitch.conf** — порядок поиска
- **/etc/resolv.conf** — DNS-серверы

ПОЛЕЗНЫЕ КОМАНДЫ

```
getent hosts example.com
resolvectl query example.com
dig example.com
nslookup example.com
```

ЕСЛИ ИМЯ НЕ НАЙДЕНО

- ошибка NXDOMAIN
- timeout / нет ответа DNS
- проверь DNS, сеть и /etc/hosts

ГЛАВНАЯ ИДЕЯ: Linux ищет имя по порядку: локальные записи → настройки NSS → DNS-сервер → IP-адрес для подключения

5. Настройка hostname

Hostname - системное имя Linux-узла.

Оно отображается в приглашении shell, журналах, SSH и мониторинге.

Имя должно быть осмысленным: linux-srv01, dns01, web01.

Случайные имена вроде ubuntu или localhost усложняют сопровождение стенда.

hostnamectl: просмотр имени

Для просмотра имени используют: **hostnamectl**

Команда показывает Static hostname, Operating System, Kernel и Architecture.

Также можно быстро вывести имя командой hostname.

В отчете полезно фиксировать hostname вместе с IP-адресом.

hostnamectl: изменение имени

Для изменения имени используют: **sudo hostnamectl set-hostname linux-srv01**

После изменения проверяют: **hostnamectl**

В некоторых стендах также проверяют /etc/hosts, чтобы локальное имя разрешалось корректно.

После смены имени новые SSH-сессии и журналы будут использовать новое имя.

6. SSH: назначение

SSH - Secure Shell, защищенный протокол удаленного доступа к командной строке.

SSH позволяет администратору управлять сервером без физического доступа.

Клиент SSH запускается на рабочей станции администратора.

SSH-сервер, обычно sshd, работает на Linux-сервере.

Стандартный порт SSH - TCP 22.

СХЕМА: SSH-КЛИЕНТ, СЕРВЕР И ПОРТ 22

Безопасное удалённое управление Linux-сервером через SSH (порт 22/TCP).

SSH (Secure Shell) — это криптографический протокол для безопасного удалённого управления и передачи данных.

1. ЧТО ТАКОЕ SSH?

SSH — сетевой протокол, который позволяет:

- Безопасно подключаться к удалённому серверу
- Выполнять команды в терминале
- Передавать файлы (SCP, SFTP)
- Создавать туннели и пробрасывать порты

SSH работает поверх TCP по умолчанию через порт 22.

2. ОБЩАЯ СХЕМА СОЕДИНЕНИЯ



3. РОЛЬ ПОРТА 22

- SSH-сервер слушает входящие подключения на TCP-порту 22.
- Клиент устанавливает соединение на этот порт.
- Если порт 22 закрыт — подключение невозможно.
- Порт можно изменить в конфигурации SSH-сервера (не рекомендуется без необходимости).

Открывайте порт 22 только для доверенных IP и используйте ключи вместо паролей!

4. КАК РАБОТАЕТ SSH (упрощённо)

1. Клиент подключается к серверу: порт 22/TCP
2. Сервер отправляет свой публичный ключ
3. Клиент проверяет ключ сервера (known_hosts)
4. Происходит аутентификация (пароль или ключ)
5. Устанавливается зашифрованный канал для обмена данными

После установки соединение весь трафик (команды, пароли, файлы) шифруется.

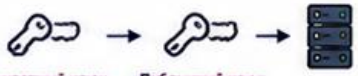
5. АУТЕНТИФИКАЦИЯ В SSH

По паролю



- Просто в использовании
- Менее безопасно
- Рекомендуется только для разового доступа

По ключу (рекомендуется)



Приватный ключ (на клиенте) — Публичный ключ (на сервере)

- Более безопасно
- Устойчиво к перебору паролей
- Удобно для автоматизации и скриптов

6. ЧТО МОЖНО ДЕЛАТЬ ПО SSH

Удалённый терминал
```
ssh user@192.168.1.10
```

Передача файлов (SCP)
```
scp file.txt user@host:/tmp/
```

Передача файлов (SFTP)
```
sftp user@host
```

Туннелирование портов
```
ssh -L 8080:localhost:80 user@host
```

Выполнение удалённых команд
```
ssh user@host "uptime"
```

7. БЕЗОПАСНОСТЬ

- Используйте ключевую аутентификацию
- Отключите вход по паролю (PasswordAuthentioction no)
- Ограничьте доступ по IP (файрвол/UFW)
- Не используйте стандартный порт 22 (по возможности)
- Обновляйте сервер и SSH-пакеты
- Включите ограничение попыток входа (Fail2ban)

Никогда не используйте root для входа по SSH! Создайте обычного пользователя с sudo.

8. ПРИМЕРЫ КОМАНД

Подключение
```
ssh user@192.168.1.10
ssh -p 2222 user@192.168.1.10
ssh -i ~/.ssh/id_rsa user@host
```

Генерация ключей (клиент)
```
ssh-keygen -t ed25519 -C "me@host"
ls ~/.ssh/
cat ~/.ssh/id_ed25519.pub
```

Копирование ключа на сервер
```
ssh-copy-id user@192.168.1.10
```
или вручную:
```
cat id_ed25519.pub | ssh user@host "mkdir -p ~/.ssh && cat >> ~/.ssh/authoriced_keys"
```

Передача файлов
```
scp file.txt user@host:/tmp/
scp -r dir/ user@host:/backup/
sftp user@host
```

9. ПРОВЕРКА СОЕДИНЕНИЯ И ПОРТА

Проверка SSH-сервиса на сервере	`sudo systemctl status sshd`
Проверка слушающего порта	`sudo ss -tlnp | grep :22`
Проверка подключения с клиента	`nc -vz 192.168.1.10 22` или: `telnet 192.168.1.10 22`
Проверка фаервола (UFW)	`sudo ufw status`

10. ТИПИЧНЫЕ ПРОБЛЕМЫ

Connection refused
SSH не запущен или порт закрыт

Permission denied (publickey,password)
Неверный пароль или ключ

Host key verification failed
Изменился ключ сервера (проверить known_hosts)

Operation timed out
Порт 22 закрыт фаерволом или сеть недоступна

СОВЕТЫ

- Используйте ключи SSH
- Ограничивайте доступ по IP
- Мониторьте логи: /var/log/auth.log или journalctl -u sshd
- Делайте резервные копии перед изменением конфигурации

ВАЖНО: Порт 22/TCP должен быть открыт на сервере и в сетевом экране.

Подключение по SSH

Базовый формат подключения: ssh пользователь@адрес_сервера.

Пример: **ssh student@192.168.10.20**.

При первом подключении клиент показывает fingerprint сервера.

Fingerprint нужно осознанно принять, если это действительно нужный сервер.

После успешного входа команды выполняются на удаленном сервере.

Проверка SSH-службы на сервере

Состояние службы проверяют через:
systemctl status ssh.
Если служба не запущена, используют:
sudo systemctl start ssh.
Автозагрузку проверяют через:
systemctl is-enabled ssh.
Если нужно включить и запустить сразу:
sudo systemctl enable --now ssh.

Проверка порта SSH

Даже active-служба должна слушать сетевой порт.

Проверка порта: ss -tlnp | grep ':22'.

Ключ -t показывает TCP, -l только listening, -n числовые адреса, -p процесс.

Если порт 22 не слушается, клиент не сможет подключиться по SSH.

7. SSH-ключи

SSH-ключи - способ аутентификации без передачи пароля по сети.

Пара ключей состоит из закрытого и открытого ключа.

Закрытый ключ остается у администратора и не передается другим людям.

Открытый ключ копируется на сервер в authorized_keys.

Сервер проверяет, что клиент владеет соответствующим закрытым ключом.

СХЕМА: АУТЕНТИФИКАЦИЯ ПО SSH-КЛЮЧУ

Безопасный вход на сервер без пароля с использованием пары ключей.

Аутентификация по SSH-ключу безопаснее пароля: приватный ключ остаётся у вас и не передаётся по сети.

1. ЧТО ТАКОЕ SSH-КЛЮЧИ?

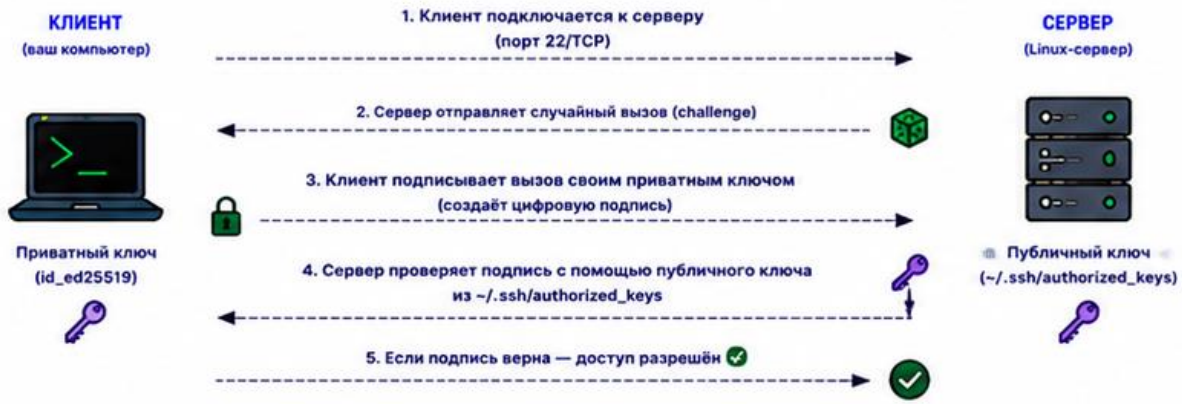
SSH использует пару ключей:

Приватный ключ (private key)
Хранится только у клиента
(на вашем компьютере).
Никому не передаётся!

Публичный ключ (public key)
Можно передавать и хранить на сервере
(~/.ssh/authorized_keys).
Сервер использует его для проверки.

Только владелец приватного ключа может пройти аутентификацию на сервере.

2. КАК РАБОТАЕТ АУТЕНТИФИКАЦИЯ ПО SSH-КЛЮЧУ



3. ПРЕИМУЩЕСТВА

- Нет передачи пароля по сети
- Защита от перебора паролей (brute force)
- Можно использовать парольную фразу (passphrase) для дополнительной защиты
- Удобство и автоматизация (скрипты, CI/CD и т.д.)

ВАЖНО!

- Никогда не передавайте приватный ключ!
- Храните приватный ключ в безопасности.
- Используйте парольную фразу для ключа.

4. ГЕНЕРАЦИЯ ПАРЫ КЛЮЧЕЙ (НА КЛИЕНТЕ)

```
$ ssh-keygen -t ed25519 -C "user@example.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/user/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/user/.ssh/id_ed25519
Your public key has been saved in /home/user/.ssh/id_ed25519.pub
```

Файлы после генерации:

```
~/.ssh/id_ed25519        # приватный ключ (600)
~/.ssh/id_ed25519.pub    # публичный ключ (644)
```

5. КОПИРОВАНИЕ ПУБЛИЧНОГО КЛЮЧА НА СЕРВЕР

Способ 1: ssh-copy-id (рекомендуется)

```
$ ssh-copy-id user@server_ip
```

Публичный ключ будет добавлен в ~/.ssh/authorized_keys

Способ 2: вручную

1. `$ cat ~/.ssh/id_ed25519.pub`
   Скопируйте вывод (ключ)
2. На сервере:
   ```
   $ mkdir -p ~/.ssh
   $ chmod 700 ~/.ssh
   $ nano ~/.ssh/authorized_keys
   ```
   Вставьте ключ в файл
3. ```
   $ chmod 600 ~/.ssh/authorized_keys
   $ chown -R user:user ~/.ssh
   ```

6. НАСТРОЙКА SSH-СЕРВЕРА (РЕКОМЕНДУЕТСЯ)

/etc/ssh/sshd_config

```
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
PasswordAuthentication no   # отключить вход по паролю
ChallengeResponseAuthentication no
```

Применить изменения:

```
$ sudo systemctl reload sshd
или
$ sudo systemctl restart sshd
```

7. ПОДКЛЮЧЕНИЕ БЕЗ ПАРОЛЯ

```
$ ssh user@server_ip
```

- Если всё настроено правильно —
- вход выполнится без запроса пароля!



8. КЛЮЧ С ПАРОЛЬНОЙ ФРАЗОЙ (РЕКОМЕНДУЕТСЯ)

При генерации укажите парольную фразу.
Она будет запрашиваться при использовании ключа.

```
$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/user/.ssh/id_ed25519:
Identity added: /home/user/.ssh/id_ed25519 (user@example.com)
```

Агент ssh-agent хранит ключ в памяти, чтобы не вводить парольную фразу каждый раз.

9. УСТРАНЕНИЕ НЕПОЛАДОК

Проблема	Решение
Permission denied (publickey)	Проверьте права на ~/.ssh и authorized_keys
Ключ не запрашивается	Проверьте настройки sshd_config и клиента
Неверный ключ	Убедитесь, что нужный ключ загружен (ssh-add -l)
Файл не читается	Права: ~/.ssh (700), authorized_keys (600)

10. ПОЛЕЗНЫЕ КОМАНДЫ

```
$ ssh-keygen -t ed25519 -C "email"   # создать ключ
$ ssh-add ~/.ssh/id_ed25519          # добавить ключ в агент
$ ssh-add -l                         # показать загруженные ключи
$ ssh-copy-id user@server_ip         # скопировать ключ на сервер
$ ssh -v user@server_ip              # подробный вывод (отладка)
```

11. ЛУЧШИЕ ПРАКТИКИ

- Используйте ed25519 ключи
- Обязательно задавайте парольную фразу
- Отключайте вход по паролю на сервере
- Ограничивайте доступ по IP (Firewall)
- Используйте отдельные ключи для разных задач

СОВЕТ:

- Храните приватный ключ в безопасном месте
- Делайте резервные копии приватного ключа
- Используйте разные ключи для работы и автоматизации
- Отзывайте доступ, удаляя ключ из authorized_keys
- Комбинируйте с ограничением по IP и Fail2ban для максимальной защиты
```

Меры повышения безопасности SSH:

- Отказ от паролей: Переключиться на аутентификацию по ключам (Ed25519). Ключи невозможно подобрать перебором (брутфорсом).
- Запрет Root-доступа: В /etc/ssh/sshd_config выставить PermitRootLogin no. Хакеры не смогут зайти под именем суперпользователя напрямую.
- Отключение парольного входа: В файле конфигурации задать PasswordAuthentication no (строго после проверки работы ключей!).
- Смена стандартного порта: Изменить порт со стандартного 22 на любой случайный (например, 2222 или 49152), чтобы скрыться от 99% автоматических ботов-сканеров.
- Ограничение в UFW: Разрешить доступ к SSH-порту в файрволе только для конкретных доверенных IP-адресов.
- Установка Fail2ban: Защитить порт утилитой, которая автоматически блокирует в файрволе IP-адреса после нескольких неудачных попыток входа.

Создание SSH-ключа

Современный тип ключа для учебной практики: ed25519.

Пример создания: ssh-keygen -t ed25519 -C "student@lab".

Закрытый ключ обычно сохраняется в ~/.ssh/id_ed25519.

Открытый ключ сохраняется в ~/.ssh/id_ed25519.pub.

Для закрытого ключа желательно задать passphrase.

Копирование открытого ключа

Удобный способ установки ключа: ssh-copy-id student@192.168.10.20.

Команда добавляет открытый ключ в ~/.ssh/authorized_keys на сервере.

После этого проверяют вход: ssh student@192.168.10.20.

Если ssh-copy-id недоступен, открытый ключ можно добавить вручную, но нужно сохранить права файлов.

Права каталога ~/.ssh

Каталог ~/.ssh должен быть доступен только владельцу.

Типичные права каталога: chmod 700 ~/.ssh.

Файл authorized_keys обычно имеет права: chmod 600 ~/.ssh/authorized_keys.

Если права слишком открытые, sshd может отказаться принимать ключи.

Владелец файлов должен совпадать с пользователем учетной записи.

8. Базовая настройка sshd_config

sshd_config - основной файл настройки SSH-сервера.

Обычно он расположен в /etc/ssh/sshd_config.

Перед правкой делают резервную копию.

Изменения SSH опасны: ошибка может заблокировать удаленный доступ.

Старую SSH-сессию не закрывают, пока новая проверка входа не выполнена.

Резервная копия sshd_config

Перед правкой используют:
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.

Копия нужна для быстрого возврата к рабочей конфигурации.

После копии файл открывают в редакторе, например sudo nano /etc/ssh/sshd_config.

Менять лучше один смысловой блок за раз.

Проверка синтаксиса конфигурационного файла перед перезагрузкой:

sudo sshd -t

Root login

Параметр PermitRootLogin управляет входом root по SSH.

Для базовой безопасности root login обычно запрещают.

Типичное значение: PermitRootLogin no.

Администратор входит обычным пользователем и повышает права через sudo.

Перед запретом root нужно убедиться, что обычный пользователь имеет sudo-доступ.

Парольный вход и ключи

PasswordAuthentication управляет входом по паролю.

PubkeyAuthentication управляет входом по SSH-ключам.

Отключать парольный вход можно только после успешной проверки входа по ключу.

Типичная безопасная логика: сначала настроить ключи, затем проверить, затем ограничивать парольный вход.

Проверка конфигурации SSH

Перед перезапуском SSH проверяют синтаксис: sudo sshd -t

Если вывода нет, синтаксис обычно корректен.

После изменения применяют: sudo systemctl reload ssh или sudo systemctl restart ssh

Затем открывают новую сессию SSH и проверяют вход.

Старую рабочую сессию закрывают только после успешной проверки новой.

9. Firewall через ufw

Сетевой экран (файрвол) в Linux контролирует входящий и исходящий сетевой трафик на основе заданных правил безопасности.

В современных дистрибутивах Debian и Ubuntu архитектура файрвола разделена на низкоуровневый движок ядра и удобные пользовательские утилиты.

Эволюция подсистем ядра: iptables и nftables (Для ознакомления)

Все сетевые экраны в Linux работают на уровне ядра операционной системы. Инструменты командной строки лишь управляют этими ядерными модулями.

Исключенный из стандартов: iptables

Что это: Исторический стандарт сетевого экрана Linux, использовавшийся более 20 лет.

Как устроен: Делит трафик на таблицы (filter, nat, mangle) и цепочки (INPUT, OUTPUT, FORWARD).

Минусы: Сложный, громоздкий синтаксис. Плохо масштабируется на современных многоядерных процессорах и при обработке гигантских списков IP-адресов. Раздельные утилиты для IPv4 (iptables) и IPv6 (ip6tables).

Современный статус: Устарел. В современных Debian/Ubuntu заменен на слой совместимости iptables-nft, который переводит старые команды в правила для нового движка nftables.

Современный стандарт: nftables (утилита nft)

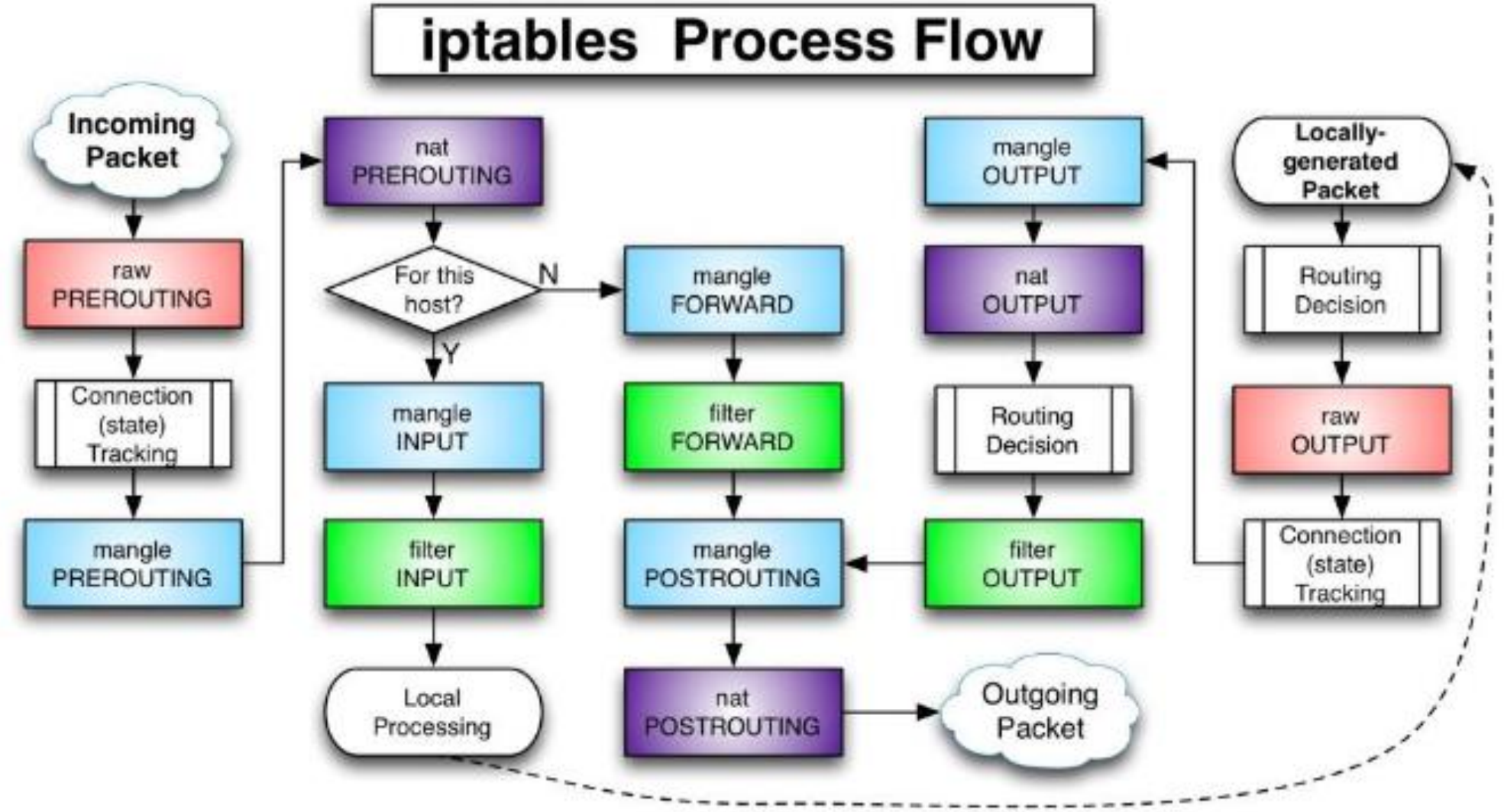
Что это: Актуальный и очень быстрый движок ядра, пришедший на смену iptables.

Плюсы:

- Объединяет IPv4, IPv6, ARP и сетевые мосты в единый контекст.
- Работает значительно быстрее за счет встроенной виртуальной машины JIT-компиляции прямо в ядре.
- Синтаксис стал чище и логичнее, но остался низкоуровневым (инженерным).

Применение: Используется по умолчанию «под капотом» в Debian 10+ и Ubuntu 20.04+.

Движение трафика в Linux:
Цепочки и таблицы файрвола (iptables / nftables)



Подробнее про UFW (Uncomplicated Firewall)

UFW (Простой файрвол) — это дружелюбная текстовая надстройка (фронтенд) над системным движком. Она создана разработчиками Ubuntu, чтобы избавить администратора от необходимости писать сложные системные таблицы ядра, позволяя управлять защитой сервера простыми человекочитаемыми командами.

Где используется: Установлен по умолчанию в Ubuntu. В Debian его можно легко добавить командой sudo apt install ufw.

Базовая логика UFW:

- По умолчанию после активации UFW включает безопасный режим: блокировать всё входящее (deny incoming) и разрешать всё исходящее (allow outgoing).
- То есть изнутри сервера в Интернет выйти можно, а снаружи к серверу никто не подключится, пока вы явно не откроете порт.

Шпаргалка по основным командам UFW

Перед началом настройки сетевого экрана на удаленном сервере критически важно сначала разрешить порт SSH, иначе вас мгновенно отключит от сессии без возможности вернуться.

Управление состоянием службы:

- sudo ufw status — Проверить текущий статус (покажет Status: active или inactive и список правил).
- sudo ufw allow ssh (или порт 22/tcp) — Самый первый шаг! Разрешить входящие подключения для управления сервером.
- sudo ufw enable — Включить файрвол (правила начнут действовать, добавится в автозагрузку).
- sudo ufw disable — Полностью отключить сетевой экран.
- sudo ufw reset — Сбросить все настройки к заводским.

Создание правил (Открытие портов):

- sudo ufw allow 80/tcp — Открыть порт 80 (HTTP) строго для TCP-трафика.
- sudo ufw allow 53 — Открыть порт 53 (DNS) сразу для протоколов TCP и UDP.
- sudo ufw allow from 192.168.1.50 — Разрешить любые подключения, но строго с доверенного IP-адреса.
- sudo ufw allow from 10.0.0.0/24 to any port 3306 — Разрешить доступ к базе данных (порт 3006) только для компьютеров из локальной подсети.

Закрытие и удаление правил:

- sudo ufw deny 21/tcp — Явно запретить входящий трафик на порт FTP (21).
- sudo ufw delete allow 80/tcp — Удалить ранее созданное разрешающее правило для порта 80.
- sudo ufw status numbered → sudo ufw delete 3 — Удаление по номеру: сначала выводится список правил с индексами, а затем одной командой удаляется строка под номером 3.

СХЕМА: БАЗОВЫЕ ПРАВИЛА UFW

UFW (Uncomplicated Firewall) — простой интерфейс для управления iptables.

UFW по умолчанию блокирует все входящие соединения и разрешает исходящие. Правила обрабатываются сверху вниз.

1. ЧТО ТАКОЕ UFW?

UFW — это простой способ управления брандмауэром в Linux.
Он работает поверх iptables и упрощает создание правил.



- Простой синтаксис
- Профили приложений
- IPv4 и IPv6
- Логирование
- Интеграция с systemd

2. СОСТОЯНИЕ UFW

Проверка статуса:

```
sudo ufw status verbose
```

Варианты состояний:

active	Брандмауэр включён и работает
inactive	Брандмауэр выключен
not active	Брандмауэр не загружен

Управление UFW:

```
sudo ufw enable    # включить и загрузить при старте системы
sudo ufw disable   # выключить
sudo ufw reload    # перезагрузить правила
sudo ufw reset     # сбросить все правила (к значениям по умолчанию)
```

3. ПОЛИТИКИ ПО УМОЛЧАНИЮ

Политика определяет, что делать с пакетами, для которых нет явного правила.

Проверка политик:

```
sudo ufw status verbose
```

Тип трафика	По умолчанию
Входящий (incoming)	deny (reject)
Исходящий (outgoing)	allow (accept)

Изменение политик по умолчанию:

```
sudo ufw default allow incoming
sudo ufw default deny  incoming
sudo ufw default allow outgoing
sudo ufw default deny  outgoing
```

4. ОСНОВНЫЕ ПРАВИЛА

Разрешить входящее соединение:

```
sudo ufw allow <порт>[/протокол]
```

Пример: `sudo ufw allow 22/tcp`

Запретить входящее соединение:

```
sudo ufw deny <порт>[/протокол]
```

Пример: `sudo ufw deny 23/tcp`

Разрешить исходящее соединение:

```
sudo ufw allow out <порт>[/протокол]
```

Пример: `sudo ufw allow out 80/tcp`

Запретить исходящее соединение:

```
sudo ufw deny out <порт>[/протокол]
```

Пример: `sudo ufw deny out 25/tcp`

Протокол можно не указывать (по умолчанию tcp).

5. ПОЛЕЗНЫЕ ПРИМЕРЫ

SSH (порт 22)	`sudo ufw allow 22/tcp`
HTTP (порт 80)	`sudo ufw allow 80/tcp`
HTTPS (порт 443)	`sudo ufw allow 443/tcp`
DNS (порт 53)	`sudo ufw allow 53/udp`
Диапазон портов	`sudo ufw allow 1000:2000/tcp`
По IP-адресу	`sudo ufw allow from 192.168.1.10`
По подсети	`sudo ufw allow from 192.168.1.0/24`
С описанием (комментарий)	`sudo ufw allow 22/tcp comment 'SSH access'`

6. ПРОФИЛИ ПРИЛОЖЕНИЙ

UFW умеет управлять правилами для популярных приложений.

Список приложений:

```
sudo ufw app list
```

Разрешить приложение:

```
sudo ufw allow <имя_приложения>
```

Пример: `sudo ufw allow OpenSSH`

Удалить приложение:

```
sudo ufw delete allow <имя_приложения>
```

Пример вывода ufw app list:

```
Available applications:
  OpenSSH
  Apache
  Nginx Full
  Nginx HTTP
  Nginx HTTPS
  Samba
  ...
```

7. ПРОСМОТР И УДАЛЕНИЕ ПРАВИЛ

Список правил:

```
sudo ufw status numbered
```

№	To	Action	From
[1]	22/tcp	ALLOW IN	Anywhere
[2]	80/tcp	ALLOW IN	Anywhere
[3]	443/tcp	ALLOW IN	Anywhere
[4]	22/tcp (v6)	ALLOW IN	Anywhere (v6)

Удалить правило по номеру:

```
sudo ufw delete <номер_правила>
```

Удалить правило (по параметрам):

```
sudo ufw delete allow 22/tcp
```

8. ЛОГИРОВАНИЕ

Включить логирование:

```
sudo ufw logging on
```

Отключить логирование:

```
sudo ufw logging off
```

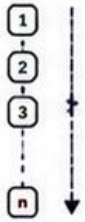
Уровни логирования:

- `low` – логировать только отклонённые пакеты
- `medium` – логировать разрешённые и отклонённые (по умолчанию)
- `high` – логировать всё

Пример:

```
sudo ufw logging high
```

9. ПОРЯДОК ОБРАБОТКИ ПРАВИЛ



Правила обрабатываются сверху вниз.
Применяется первое совпавшее правило.

- ALLOW – пакет пропускается
- DENY – пакет блокируется
- Если нет совпадений – действует политика по умолчанию

10. ПРОВЕРКА СОЕДИНЕНИЙ

Проверка из другого хоста:

```
telnet <IP_сервера> <порт>

nc -vz <IP_сервера> <порт>

nmap -p <порт> <IP_сервера>
```

Пример:

```
$ nc -vz 192.168.1.10 22
Connection to 192.168.1.10 22 port [tcp/ssh] succeeded!

$ nc -vz 192.168.1.10 23
nc: connect to 192.168.1.10 port 23 (tcp) failed: Connection refused
```

11. ПРОВЕРКА СТАТУСА

Краткий статус:	`sudo ufw status`
Детальный статус:	`sudo ufw status verbose`
Статус по IPv6:	`sudo ufw status ipv6`
Номера правил:	`sudo ufw status numbered`

12. ВАЖНЫЕ ЗАМЕТКИ

- Всегда убедитесь, что доступ по SSH разрешён, чтобы не заблокировать себя.
- При изменении правил проверяйте из другого подключения.
- После изменения правил:
  – они применяются сразу;
  – сохраняются в `/etc/ufw/user.rules`.

Проверка состояния ufw

Состояние firewall проверяют через: sudo ufw status verbose.

Если ufw inactive, правила могут быть настроены, но не применяются.

verbose показывает политику по умолчанию и разрешенные правила.

Перед включением нужно убедиться, что есть правило для SSH.

Разрешение SSH и HTTP

Разрешить SSH можно профилем: sudo ufw allow SSH

Можно указать порт явно: sudo ufw allow 22/tcp

Для веб-сервера разрешают HTTP: sudo ufw allow 80/tcp.

Для HTTPS используют: sudo ufw allow 443/tcp.

После добавления правил снова проверяют sudo ufw status verbose.

Включение ufw без потери доступа

Правильный порядок: сначала разрешить SSH, затем включать firewall.

Пример: sudo ufw allow SSH

Затем: sudo ufw enable

После включения проверяют текущую SSH-сессию и открывают новую.

Если новая сессия не открывается, старую сессию не закрывают.

Профили приложений UFW

Профиль UFW — это готовое описание сетевых портов, необходимых приложению.

Профили хранятся в каталоге:

/etc/ufw/applications.d/

UFW читает эти файлы и позволяет использовать имя приложения вместо ручного указания порта. Ubuntu Manpages

Например:

sudo ufw allow OpenSSH

вместо:

sudo ufw allow 22/tcp

Преимущество профиля: администратору не обязательно помнить номера всех портов приложения.

Просмотр доступных профилей

Посмотреть профили, которые известны UFW:
sudo ufw app list

Пример вывода:
Available applications:
  OpenSSH

Набор профилей зависит от установленных в системе программ.
Посмотреть информацию о конкретном профиле:
sudo ufw app info OpenSSH

UFW показывает описание профиля и порты, которые он использует.
Команды app list и app info являются штатными командами UFW.

Использование профилей в правилах

Разрешить приложение:
sudo ufw allow OpenSSH

Запретить:
sudo ufw deny OpenSSH

Удалить ранее созданное правило:
sudo ufw delete allow OpenSSH

Можно разрешить приложение только для определённой сети:
sudo ufw allow from 192.168.10.0/24 to any app OpenSSH

Здесь:
192.168.10.0/24
↓
разрешённая сеть

OpenSSH
↓
профиль приложения

В расширенном синтаксисе UFW app <имя> используется вместо указания порта.

Как устроен профиль UFW

Профиль представляет собой обычный текстовый файл в формате, похожем на INI.
Упрощённый пример:

```
[MyWebServer]
title=My Web Server
description=Учебный веб-сервер
ports=8080/tcp
```

После создания профиля:

```
sudo ufw app list
```

может показать:

```
MyWebServer
```

Тогда правило можно добавить по имени:

```
sudo ufw allow MyWebServer
```

Для применения изменений существующего профиля предусмотрена команда:

```
sudo ufw app update MyWebServer
```

Официальный синтаксис профиля содержит поля title, description и ports; один профиль также может описывать несколько портов и протоколов.

Ограничение SSH по сети

SSH лучше открывать не всем, а только сети управления.

Пример: sudo ufw allow from 192.168.10.0/24 to any port 22 proto tcp.

Так сервер принимает SSH только из указанной подсети.

Правило должно соответствовать адресному плану стенда.

После изменения проверяют доступ с разрешенного и запрещенного клиента.

Удаление ошибочного правила ufw

Список правил с номерами: sudo ufw status numbered

Удаление правила по номеру: sudo ufw delete 2

Номер нужно проверять перед удалением, потому что после удаления список меняется.

После удаления снова выполняют sudo ufw status numbered.

Итоговая диагностика сети и SSH

- Интерфейс: ip link и ip addr.
- Маршрут: ip route и ip route get.
- Доступность gateway: ping -c 4 адрес_gateway.
- DNS: resolvectl query или dig +short.
- SSH и firewall: systemctl status ssh, ss -tlnp, sudo ufw status verbose.

Типовые ошибки

- Проверять DNS до проверки IP-связности.
- Иметь правильный IP, но забыть default gateway.
- Отключить парольный SSH-вход до проверки ключей.
- Включить ufw до разрешения SSH.
- Считать status ssh достаточным, не проверив порт и клиентское подключение.

Итоги лекции

- Сеть Linux проверяется по уровням: интерфейс, IP, маршрут, DNS, TCP-порт.
- SSH - основной инструмент удаленного администрирования Linux-сервера.
- SSH-ключи повышают безопасность, если закрытый ключ защищен и не передается другим.
- sshd_config меняют осторожно: копия, проверка синтаксиса, reload/restart, новая сессия.
- ufw включают только после разрешения нужного удаленного доступа.

Контрольные вопросы

1. Что показывает ip link, а что ip addr?
2. Для чего нужен default route?
3. Как отличить проблему DNS от проблемы маршрутизации?
4. Что такое SSH-клиент и SSH-сервер?
5. Чем открытый SSH-ключ отличается от закрытого?
6. Почему нельзя сразу отключать PasswordAuthentication?
7. Что нужно сделать перед sudo ufw enable?
8. Как проверить, что SSH действительно слушает порт 22?